# Feature Extraction vs Fine-Tuning, Layer Freezing

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology




KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 5, Day 32, Self-study

20 October 2026

# Table of Contents


1. Where This Fits
2. Learning Outcomes
3. The Freeze Line Inside Autograd
4. Four Traps That Fail Silently
5. Comparing Fairly, Fitting the Budget
6. Other Families, and the Contest Checklist
7. Olympiad Practice
8. Check Yourself
9. Limitations and What Comes Next
10. Summary
11. References

# From the Lecture

- **In the lectures:** Day 29 took a pretrained ResNet-18 apart (encoder, head, the weights file) and trained a new head on a frozen encoder. Day 30 named the two ways, feature extraction (linear probing) and fine-tuning, wrote the update rule with one learning rate per parameter group, drew the freeze line through ResNet-18's six groups, thawed `layer4` with two learning rates, built the 2.6 ladder and gradual unfreezing, and showed when fine-tuning pays (Kornblith et al.) and when it forgets (Kumar et al.). Lab 1 ran the three strategies on one split.
- **This deck adds:** what `requires_grad` does inside autograd and what each freeze line costs in compute and memory, measured; the three switches people confuse (`requires_grad`, `torch.no_grad()`, `eval()`); BatchNorm under a frozen encoder; four traps that fail *silently*, each with its symptom, cause and fix; a fair comparison protocol and the budget arithmetic of a timed contest; learning-rate schedules for fine-tuning; freezing in other model families; and a contest checklist.
- **How to use it:** every frame stands alone and every number was measured or quoted. Run each code frame on Day 30's Lab 1 notebook, break it on purpose where the deck says "trap", and answer the check at the end before turning to the answers.

# Why Go Deeper

## Where the lecture's version breaks

- You set `requires_grad = True` on `layer4`, train five more epochs, and nothing changes: the optimizer was built before the thaw and never heard of those weights. No error, no warning, a wasted budget.
- The frozen run reached 0.89; you thaw everything with the same `lr=1e-3` and the validation accuracy *falls* to 0.6 within an epoch. Day 30's math said why; the deck shows what it looks like and how to catch it in one epoch.
- A task gives you 10 minutes of GPU (IOAI 2026 at-home) or 2.5 minutes to evaluate (Polish Olympiad 2026). Which freeze line fits? Thawing everything triples the cost of a step; the arithmetic decides before the first epoch runs.
- `load_state_dict` stops with `size mismatch for fc.weight`: the checkpoint and the model disagree about the head.

## After this deck you can

choose a freeze line from the budget, set it up without any of the silent traps, keep BatchNorm where you want it, compare strategies fairly on one split, pick a learning-rate schedule, and do the same in a ViT or a Hugging Face model.

# ⇒ Learning Outcomes

By the end of this deck, you will be able to:

- **Explain** what `requires_grad = False` changes inside autograd, and tell it apart from `torch.no_grad()` and `model.eval()`.
- **Compute** what a freeze line costs: trainable parameters, FLOPs per step and activation memory, from measured numbers, and fit a run into a time budget.
- **Set up** a freeze line without the silent traps: the optimizer's parameter groups, BatchNorm's running statistics, the device of a new head, checkpoints with a different head.
- **Compare** feature extraction and fine-tuning fairly on one split, read the four curve patterns, and choose a learning-rate schedule.
- **Freeze** the right parts of a ViT, an EfficientNet, a timm model and a Hugging Face model by reading `print(model)`.
- **Plan** a fine-tuning solution to a timed Olympiad task: freeze line, head, learning rates, epochs, checks.

# The Freeze Line **Inside Autograd**

# What `requires_grad = False` Changes

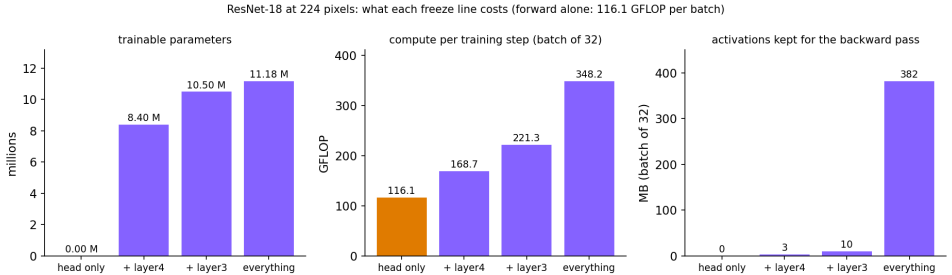
### Definition

Every tensor carries a flag, `requires_grad`. In the forward pass autograd **records** an operation only when at least one input requires a gradient; otherwise the output is produced *without* a record. The backward pass walks the records from the loss back to the tensors that require gradients, and stops where the records stop.

| | **frozen stage** (`requires_grad = False`) | **thawed stage** |
|---|---|---|
| forward pass | runs as usual, same output | runs as usual |
| autograd record | none for its operations | one node per operation |
| activations kept for backward | no | yes (for the weight gradients) |
| `.grad` after `backward()` | `None` | a tensor the shape of the weight |
| updated by the optimizer | never | if the optimizer holds it |

- The autograd notes' rule: "if there's a single input to an operation that requires gradient, its output will also require gradient". So the backward of a thawed `layer4` stops at its input: nothing below is visited or stored.

# What Each Line Costs: Measured on ResNet-18




Source: the published ResNet-18 architecture with a 5-class head, batch of 32 at 224 pixels: forward FLOPs from `torch.utils.flop_counter`, backward counted as twice the forward of every thawed stage, activations as the outputs of the thawed stages (a lower bound); `scripts/day32_freezing_figures.py`.

- The forward pass costs **116 GFLOP** per batch whatever the line. The backward adds about twice the forward of each thawed stage: `layer4` 53, `layer3` another 53, everything 232. A full fine-tuning step costs **3.0 times** a head-only step.
- Memory follows the same line: with everything thawed, autograd keeps the stem's $64 \times 112 \times 112$ maps for 32 photos, 100 MB each copy; with `layer4` alone, 3 MB.

# In Code: One Function for Every Freeze Line

```python
STAGES = ["conv1", "bn1", "layer1", "layer2", "layer3", "layer4", "fc"]   # bottom to top

def freeze_until(model, first_trainable):
    """Freeze every stage below first_trainable; thaw it and everything above."""
    thaw = STAGES[STAGES.index(first_trainable):]
    for name, param in model.named_parameters():
        param.requires_grad = name.split(".")[0] in thaw   # "layer4.1.conv2.weight" -> "
            layer4"
    return [p for p in model.parameters() if p.requires_grad]   # what the optimizer gets

for line in ["fc", "layer4", "layer3", "conv1"]:
    params = freeze_until(model, line)
    print(line, sum(p.numel() for p in params))          # 2565 ... 11179077
```

- `named_parameters()` yields `("layer4.1.conv2.weight", tensor)`: the first dotted part is the stage. One function, one argument, every line of the lecture; the printed counts are the lecture's four numbers.
- The returned list is the optimizer's argument: `torch.optim.Adam(params, lr=...)`, or two groups built from `model.fc.parameters()` and the rest. Hand the optimizer *only* what moves; the next section shows what goes wrong otherwise.

# Three Switches People Confuse

| Switch | What it turns off | When |
|---|---|---|
| `param.requires_grad = False` | gradients *for that parameter*, for good, in every forward pass | freezing a stage for training |
| `with torch.no_grad():` | autograd recording *inside the block*, for every tensor | validation and prediction: faster, nothing kept; never around the training forward pass |
| `model.eval()` | the *training behaviour* of BatchNorm (uses running statistics) and Dropout (off); gradients untouched | validation and prediction, paired with `model.train()` at the start of every epoch |

- Three independent switches: a frozen encoder in `train()` mode still updates its BatchNorm statistics; a model in `eval()` mode still computes gradients; `no_grad()` still runs Dropout in `train()` mode.
- The docs: `no_grad` is "useful for inference, when you are sure that you will not call `Tensor.backward()`"; `eval()` "has any effect only on certain modules", BatchNorm and Dropout among them.

# BatchNorm Under a Frozen Encoder

## What moves although nothing is trainable

A `BatchNorm2d` layer holds **weights** (`weight`, `bias`: frozen by `requires_grad`) and **buffers** (`running_mean`, `running_var`: updated in `train()` mode from every batch, "with a default momentum of 0.1"). ResNet-18 has 20 such layers, 9,620 running numbers.

- With the encoder frozen, each batch still nudges the statistics towards *your* photos: the features under the head shift every step. Usually harmless; with small batches or photos unlike ImageNet's it can move the validation accuracy up and down between epochs.

```
def set_bn_eval(module):
    if isinstance(module, nn.BatchNorm2d):
        module.eval()                # use
            the stored statistics

for epoch in range(epochs):
    model.train()                    #
        resets every layer to train mode
    for name, child in model.
        named_children():
        if name != "fc":             # the
            frozen part
            child.apply(set_bn_eval)
    for xb, yb in train_loader:
        ...
```

- `model.train()` switches BatchNorm back on, so the fix goes *after* it, every epoch. Everything thawed: leave BatchNorm in train mode; partly thawed: freeze the statistics of the frozen stages only.

# Four Traps **That Fail Silently**

# Trap 1: Thawed, but Not in the Optimizer

```
opt = Adam(model.fc.parameters())
...   # 3 epochs, head only
for param in model.layer4.parameters():
    param.requires_grad = True  # thaw
...   # 3 more epochs: nothing moves

# the check
w = model.layer4[1].conv2.weight
before = w.detach().clone()
train_one_epoch(model, loader, opt)
print(torch.equal(before, w))  # True
```

## Symptom

The trainable count went up, the gradients are computed (`.grad` is a tensor), the epoch takes longer, and the validation accuracy is exactly the frozen run's.

## Cause and fix

An optimizer updates only the parameters it was *given*; `requires_grad` is not a subscription. Either `opt.add_param_group(...)` with `model.layer4.parameters()` and its own `lr`, or build a new optimizer with both groups. Then run the check: it must print `False`.

# Trap 2: Wrong Device, and no_grad Around Training

```
model = resnet18(weights=...).to(device)
model.fc = nn.Linear(512, 5)   # on the CPU
out = model(xb.to(device))
# RuntimeError: Expected all tensors to be
# on the same device, but found at least
# two devices, cuda:0 and cpu!
```

- **Cause:** `.to(device)` moved the model; the head came later. **Fix:** replace the head first, then `model = model.to(device)`.

```
with torch.no_grad():      # "faster: frozen
    "
    out = model(xb)
loss = criterion(out, yb)
loss.backward()
# RuntimeError: element 0 of tensors does
# not require grad and does not have a
    grad_fn
```

- **Cause:** the block switched autograd off for the head too; the loss has no record to walk back. **Fix:** `no_grad` only around validation; a frozen encoder records nothing anyway.

One habit for both

Create, replace, then move; and let the flags decide what autograd records, never a blanket `no_grad` around training.

# Trap 3: The Checkpoint and the Model Disagree About the Head

```
model = resnet18(weights=None)                    # the architecture, random weights
model.load_state_dict(torch.load("best.pt", map_location=device))
# RuntimeError: Error(s) in loading state_dict for ResNet:
#   size mismatch for fc.weight: copying a param with shape torch.Size([5, 512])
#   from checkpoint, the shape in current model is torch.Size([1000, 512]).
```

- **Cause:** `best.pt` was saved with your 5-class head; a fresh `resnet18()` has the 1,000-class one. `load_state_dict` checks every key and shape.
- **Fix:** rebuild the model exactly as trained, *then* load: `model.fc = nn.Linear(512, 5)` before `load_state_dict`. Same order on the grader's machine (the Polish Olympiad's `Solution` class, NEOAI's fixed inference function).
- **The opposite case,** ImageNet weights into a model whose head you already replaced: `strict=False` skips the two `fc` tensors and lists them in `missing_keys`; print it, it must name `fc.weight` and `fc.bias` only.
- **Save less:** with a frozen encoder only the head changed; `torch.save(model.fc.state_dict(), "head.pt")` is 10 KB instead of 45 MB.

# Trap 4: The Frozen Run's Learning Rate, Kept After the Thaw

```
# phase 1: head only, lr 1e-3, val acc 0.89
for param in model.parameters():
    param.requires_grad = True
optimizer = Adam(model.parameters(), lr=1e
    -3)   # same lr
# epoch 1 after the thaw: val acc 0.61
```

## Symptom

The validation accuracy drops within one epoch of thawing and the training loss *also* jumps before falling again: the encoder was rewritten (Day 30's math: a `layer4` weight is about 0.010; Adam at $10^{-3}$ moves it by up to 10% per step).

## Fix: a group per part, and read the first epoch

- Pretrained stages $10^{-5}$ to $10^{-4}$, the head $10^{-3}$ (or the 2.6 ladder); a *new* optimizer for the new phase, the probed head left in place (LP-FT).
- Compare the first post-thaw epoch with the last frozen one: it must not be worse. If it is, halve the encoder's rates and restart from the frozen checkpoint; that is what `best.pt` is for.
- The frozen run's number is the floor every later phase must beat.

Comparing Fairly, **Fitting the Budget**

# A Fair Comparison: One Split, One Protocol

## The rule

Two strategies can be compared only when *everything else* is equal: the same training and validation photos (one shuffled index list, saved), the same transforms, the same number of epochs, the same batch size, the best epoch chosen the same way (by validation accuracy), and the same seed. Then the table has one moving part: the freeze line.

| Column | How to get it | What it tells you | Published anchor |
|---|---|---|---|
| trainable parameters | `count_trainable` | how much can adapt, and overfit | 2,565 to 11,179,077 |
| best validation accuracy | max over epochs | the score you would hand in | Kornblith: +1 pets, +32 cars |
| seconds per epoch | `time.time()` around the epoch | the budget | tutorial: 28 s frozen, 37 s all |
| weight movement | relative change per stage (Day 30, Lab 1) | how far from ImageNet it went | Kumar: LP-FT moves 10 to 100 times less |

- Day 30's Lab 1 is this protocol: three models, one `train_idx`, three epochs each. Never compare a 3-epoch frozen run with a 10-epoch fine-tuned one and call the difference "fine-tuning". Report validation, not training: the thawed model's training loss falls fastest when it memorises.

# Reading the Curves: Four Patterns

| Training loss | Validation accuracy | Reads as | Move |
|---|---|---|---|
| falls steadily | rises, then flattens | healthy; stop at the flat | keep the best epoch; try the next freeze line if time remains |
| falls to near 0 | flat or falling after a peak | memorising: too much moves for the photos | move the line up, smaller rates, more augmentation, fewer epochs |
| flat from the start | near chance ($1/k$) | nothing moves: Trap 1, or a frozen head, or a rate near 0 | print the trainable count and `optimizer.param_groups` |
| jumps up after a thaw | drops below the frozen run | forgetting: the rate rewrote the encoder (Trap 4) | restart from the checkpoint with the ladder |

- The patterns are Week 3's (Day 17 read loss curves) with one new cause for each: the freeze line. The questions to ask are always the same two: *how much moves*, and *how fast*.
- A frozen run that is already at the published number for a similar dataset (pets 91.5%, flowers 93.2%, Kornblith et al.) has little left to win: thaw only if the budget is free.

# The Budget Arithmetic of a Timed Task

## The equation

$$\text{time} = \text{epochs} \times \frac{N_{\text{train}}}{\text{batch}} \times t_{\text{step}} + \text{epochs} \times t_{\text{val}} \;\leq\; \text{budget}$$

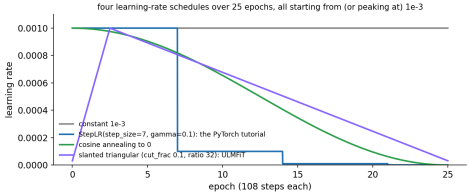
$t_{\text{step}}$ grows with the freeze line (1.45 times with `layer4`, 3.0 with everything); $t_{\text{val}}$ is one forward pass over the validation set.

### With numbers

3,453 photos, batch 32: 108 steps per epoch. A head-only step measured at 0.25 s gives epochs of 27 s frozen, 40 s with `layer4`, 80 s with everything. A **10-minute** budget: 3 frozen epochs then 3 with `layer4` cost $27 \times 3 + 40 \times 3 = 201$ s and leave 400 s for prediction and a retry; 6 epochs with everything thawed would cost 480 s.

- Measure one step first (the second batch, not the first), then plan the *whole* budget: load, train, validate, predict, write the file, retry.
- Read which clock the task measures: the Polish Olympiad limits *evaluation* (780 forward passes in 2.5 minutes, any line fits); training time is yours.

# Learning-Rate Schedules When Fine-Tuning




Source: the four formulas, drawn for 25 epochs of 108 steps; StepLR with the PyTorch tutorial's `step_size=7, gamma=0.1`; slanted triangular from Howard and Ruder (2018), `cut_frac=0.1, ratio=32`; `scripts/day32_freezing_figures.py`.

- Day 17's schedules apply to each parameter group: `StepLR(optimizer, 7, 0.1)` scales every group's `lr` together, so the ladder keeps its shape. The tutorial fine-tunes with it for 25 epochs; the ULMFiT triangle rises for 10% of the steps then decays, "to quickly converge to a suitable region ... and then refine".
- Common practice for a short budget: a constant small rate for 3 to 5 epochs, or cosine to zero over the planned epochs (`CosineAnnealingLR`); a warm-up of a few hundred steps for a ViT (Week 6 uses it for BERT).

Other Families, **and the Contest Checklist**

# Where the Head and the Stages Live in Other Models

| Model (library) | Head | Stages, bottom to top | Thaw the top stage |
|---|---|---|---|
| ResNet (torchvision) | `model.fc` | `conv1`, `layer1` to `layer4` | `model.layer4` |
| EfficientNet, MobileNet | `classifier[-1]` | `model.features[0]` to `[8]` | `features[-2:]` |
| ViT-B/16 (torchvision) | `model.heads.head` | `model.encoder.layers`, 12 blocks | `encoder.layers[-2:]` |
| any timm model | `model.get_classifier()` | `model.blocks` or `model.stages` | `model.blocks[-2:]` |
| Hugging Face (HF) | `model.classifier` | `model.vit.encoder.layer` | `vit.encoder.layer[-2:]` |

```
vit = torchvision.models.vit_b_16(weights="IMAGENET1K_V1")
for param in vit.parameters():
    param.requires_grad = False
for block in vit.encoder.layers[-2:]:                    # the last 2 of 12 blocks
    for param in block.parameters():
        param.requires_grad = True
vit.heads.head = nn.Linear(vit.heads.head.in_features, 5)
```

- Same habit in every family: `print(model)`, the last `Linear` is the head, the repeated blocks above it are the stages; freeze by name from the top. Week 6 freezes BERT's layers the HF way.

# The Contest Checklist

## Before the first epoch

1. Time one training step and one validation pass; write the budget equation with the task's clock.
2. Replace the head, then `.to(device)`; print the trainable count for the chosen line.
3. Build the optimizer from the trainable parameters only, with a group per part and the rates written down.
4. Decide BatchNorm: statistics frozen with the frozen stages, free when everything trains.
5. Save the index lists of the split and the seed.

## During and after

1. Probe first; the frozen run's validation number is the floor and its `best.pt` the fallback.
2. Thaw one stage with a *new* optimizer; compare the first epoch with the floor; retreat if it is lower.
3. Checkpoint every improvement; write the submission from the checkpoint, not from the last epoch.
4. Rebuild the model the same way before loading the checkpoint on the grader; print `missing_keys`.
5. Keep the frozen run's submission file until the thawed one beats it on validation.

- Ten lines, two minutes to run through, and every silent trap in this deck is caught by one of them.

# Layer Freezing: **Olympiad Practice**

# Practice Task: Which Clothes Are in the Picture?

## Task

Each image ($168 \times 168$ pixels) shows several pieces of clothing; the label is a vector of **ten** 0/1 entries (T-shirt, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot). Training 6,318 images, validation 702, hidden test 780. You submit the notebook: a model class, a training function, a prediction function. Score: **macro F1** over the ten labels, 0 points at F1 $\leq 0.57$, 100 at F1 $\geq 0.87$. No internet on the grader, but torchvision's pretrained **ResNet-18, 34 and 50** are stored there; evaluation on the test set within **2.5 GPU minutes**.

- **Question 1:** ten yes/no labels instead of one class: what changes in the head, the loss and the prediction, and what stays Day 29's?
- **Question 2:** 168-pixel images of clothing on plain backgrounds, far from ImageNet. Which freeze line do you start with, and which number says whether to move it?
- **Question 3:** which clock does the 2.5-minute rule measure, and what does that free you to do?

Source: III Olimpiada AI (Poland), 2026, stage 1, task 2, *Klasyfikacja wieloetykietowa* (`github.com/OlimpiadaAI/III-OlimpiadaAI`): data sizes, scoring function, allowed models and time limit from the statement.

# Solution Idea and Code

```
model = torchvision.models.resnet18(weights="IMAGENET1K_V1")    # stored on the grader
model.fc = nn.Linear(model.fc.in_features, 10)                  # ten scores, one per
    label
encoder_params = freeze_until(model, "layer3")[:-2]             # layer3 + layer4; fc is
    last
model = model.to(device)
optimizer = torch.optim.Adam([{"params": encoder_params, "lr": 1e-4},
                              {"params": model.fc.parameters(), "lr": 1e-3}])
criterion = nn.BCEWithLogitsLoss()                              # ten yes/no questions at
    once
def predict_solution(model, x):
    return (torch.sigmoid(model(x)) > 0.5).long()               # one 0/1 per label
```

1. **Recognise, then the head:** transfer learning with a freeze line to choose. A multi-label head is ten binary heads: `Linear(512, 10)`, `BCEWithLogitsLoss` (Day 16's BCE per label, y as float 0/1), each sigmoid thresholded at 0.5, tuned per label on the 702 validation images.
2. **The line:** far from ImageNet, 6,318 images: thaw `layer3` and `layer4` from the start, $10^{-4}$ on them, $10^{-3}$ on the head; the probe's F1 is the floor. ResNet-18 takes 168-pixel inputs as is.
3. **The clock:** evaluation only, 780 forward passes: seconds for any line. Budget the training yourself; rebuild before loading (Trap 3).

Layer Freezing: **Check Yourself**

# Check Yourself

Answer before turning the page.

1. **Counting.** ResNet-18 with a new 12-class head, frozen through `layer2`: how many trainable parameters, and what share of the total? (Stage counts: `layer3` 2,099,712, `layer4` 8,393,728, encoder 11,176,512.)
2. **Cost.** A head-only step takes 0.2 s on your GPU. Estimate the step with `layer4` thawed and with everything thawed, and the number of full fine-tuning epochs of 108 steps that fit in 5 minutes with 20 s of validation per epoch.
3. **Diagnosis.** After "unfreezing `layer3` and `layer4`" the trainable count printed 10,496,005, epochs got slower, and the validation accuracy stayed at exactly 0.889 for three epochs. Which trap, which one-line test, which fix?
4. **Switches.** A frozen encoder, `model.train()` at the top of each epoch, validation accuracy wandering between 0.85 and 0.90 from epoch to epoch while the training loss falls smoothly. What is moving, and what two lines stop it?

# Answers

1. Head: $512 \cdot 12 + 12 = 6{,}156$. Trainable: $2{,}099{,}712 + 8{,}393{,}728 + 6{,}156 = 10{,}499{,}596$ of $11{,}176{,}512 + 6{,}156 = 11{,}182{,}668$: 93.9%. (Frozen: the stem's 9,536, `layer1`'s 147,968, `layer2`'s 525,568.)
2. Ratios from the measured FLOPs: `layer4` thawed $168.7/116.1 = 1.45$, everything $348.2/116.1 = 3.0$: about 0.29 s and 0.6 s per step. A full epoch: $108 \times 0.6 + 20 = 85$ s; $300/85 = 3.5$, so **3 epochs**, with 45 s spare for prediction and the file.
3. Trap 1: the stages were thawed after the optimizer was built on `model.fc.parameters()`, so they get gradients but no updates; the count and the time changed, the model did not. Test: clone a `layer4` weight, train one epoch, `torch.equal` must be `False`. Fix: `add_param_group` for each thawed stage with its own `lr`, or a new optimizer with the groups.
4. BatchNorm's running mean and variance (buffers, not parameters) are updated from every training batch while the model is in `train()` mode, shifting the features under the head. After `model.train()`, apply `eval()` to the frozen stages' `BatchNorm2d` modules (`child.apply(set_bn_eval)`), every epoch; the weights were never the problem.

# Layer Freezing: **Limitations & What Comes Next**

# Limitations

- **The cost figure is arithmetic, not a stopwatch.** FLOPs count operations; a GPU's time also depends on memory traffic, batch size and the data loader, which is often the real bottleneck for a frozen encoder (Day 19). The ratios are a plan; the measured step is the truth.
- **Stages are a convenience.** ResNet's six groups are a coarse grid; the line can run through a stage (one block of `layer3`), and Lee et al. (2023) showed that for some shifts the *first* block is the one to tune. The deck's rule, from the top down, is the default, not a law.
- **Freezing is not the only lever.** Weight decay, dropout in the head, augmentation and early stopping move the same overfitting knob; Day 34 puts them together. Methods that add a few trainable weights to a frozen model (adapters, LoRA) are named here and not covered.
- **Other families differ in detail.** ViTs want a warm-up and layer-wise rates; EfficientNet's BatchNorm is as sensitive as ResNet's; Hugging Face models carry their own `train()` subtleties. The checklist holds; the names change.

The common mistake

Changing two things at once: thawing a stage *and* raising the learning rate *and* adding epochs, then crediting fine-tuning for the result, or blaming it. One moving part per experiment, the same split, the frozen run as the floor.

# What Comes Next

- **Day 33, self-study:** embedding models compared, linear probing as a measurement of a space, CLIP zero-shot and retrieval in depth: the other half of Week 5's toolbox, on cached embeddings.
- **Day 34, Olympiad practice:** small-data fine-tuning habits, each tied to a Friday-style task: what wins points, what loses them, with the sources.
- **Friday:** the Week 5 competition.
- **Week 6:** the same freeze line through a transformer: BERT's embeddings, its twelve layers and its classifier head, frozen and thawed the Hugging Face way.
- **Practice now:** open Day 30's Lab 1, replace the three models by `freeze_until` with the four lines, add the BatchNorm fix, time one step of each, and fill the budget equation for a 10-minute task.

# Layer Freezing: **Summary**

# Summary

- **Inside autograd:** `requires_grad = False` stops the recording at that stage; the backward never visits what is below the line and keeps nothing for it. A step with `layer4` thawed costs 1.45 times, everything 3.0 times a head-only step.
- **Three switches:** `requires_grad` (a parameter), `torch.no_grad()` (a block of code), `model.eval()` (BatchNorm and Dropout). BatchNorm's statistics move in `train()` mode even under a frozen encoder: set the frozen stages' `BatchNorm2d` to `eval()` after every `model.train()`.
- **Four traps:** thawed but not in the optimizer (`torch.equal` test, `add_param_group`); a head on the CPU and `no_grad` around training; a checkpoint whose head disagrees with the model (rebuild, then load); the frozen run's rate kept after the thaw.
- **Comparing and budgeting:** one split, one protocol, one moving part; four curve patterns; epochs $\times$ steps $\times$ $t_{\text{step}}$ + validation $\leq$ budget with the step measured; StepLR, cosine or the slanted triangle on every group.
- **Other families:** `print(model)`, the last `Linear` is the head, the repeated blocks above it the stages; `features[-2:]`, `encoder.layers[-2:]`, `blocks[-2:]`.
- **In a task:** probe, floor, thaw one stage with a new optimizer, checkpoint, rebuild before loading; the ten-line checklist catches every trap.

# Layer Freezing: **References & Resources**

# References & Resources


[1] PyTorch documentation, *Autograd mechanics*: “Excluding subgraphs from backward” (`requires_grad`), `torch.no_grad`, `Module.eval()` / `train()`; `BatchNorm2d` (running statistics, momentum 0.1); `Module.load_state_dict` (`strict`, `missing_keys`); `Optimizer.add_param_group`; `torch.utils.flop_counter.FlopCounterMode`.
https://docs.pytorch.org/docs/stable/notes/autograd.html

[2] PyTorch tutorials, *Transfer learning for computer vision*: `StepLR(step_size=7, gamma=0.1)`, 25 epochs, ants against bees, 28 s frozen against 37 s fine-tuned.
https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

[3] Howard, J. and Ruder, S. (2018). Universal language model fine-tuning for text classification. *ACL*. Slanted triangular learning rates (`cut_frac`, `ratio`), discriminative fine-tuning, gradual unfreezing.

[4] Kornblith, S., Shlens, J. and Le, Q. (2019). Do better ImageNet models transfer better? *CVPR*. The anchors in the comparison table. Kumar, A. et al. (2022). Fine-tuning can distort pretrained features. *ICLR*. LP-FT, feature movement. Yosinski, J. et al. (2014). How transferable are features in deep neural networks? *NeurIPS*. Lee, Y. et al. (2023). Surgical fine-tuning improves adaptation to distribution shifts. *ICLR*.

# References & Resources (cont.)


[5] III Olimpiada AI (Poland), 2026, stage 1, task 2, *Klasyfikacja wieloetykietowa* (the practice task): 6,318 / 702 / 780 images of 168 × 168 pixels, ten labels, macro F1 scored from 0.57 to 0.87, torchvision ResNet-18/34/50 available offline, evaluation within 2.5 GPU minutes. https://github.com/OlimpiadaAI/III-OlimpiadaAI IOAI 2026, At-Home Round, Task 1 (the 10-minute training budget); Individual Contest, Task 4 (ResNet-18 and `vit_tiny_patch16_224` in one task). https://github.com/IOAI-official/IOAI-2026 NEOAI 2025, Task 2, *Underfitting CV* (fixed inference function). https://www.kaggle.com/competitions/neoai-2025-underfitting-cv

[6] torchvision models documentation (ResNet, EfficientNet, MobileNet, ViT module names); Wightman, R., `timm` (`get_classifier`, `reset_classifier`); Hugging Face `transformers`, `AutoModelForImageClassification` and `base_model`. https://pytorch.org/vision/stable/models.html

[7] Figures: `slides-builder/scripts/day32_freezing_figures.py`: FLOPs and activation sizes measured once on the published ResNet-18 architecture with `FlopCounterMode` (no training), and the four learning-rate formulas. Parameter counts as on Day 30. No model was trained for this deck; every comparison number is quoted from the sources above.